



AWUPF Rediscovered: Atomic Writes to Unleash Pivotal Fault-Tolerance in SSDs

Jiyune Jeon and Jongseok Kim, *Sungkyunkwan University*;
Sam H. Noh, *Virginia Tech*; Euseong Seo, *Sungkyunkwan University*

<https://www.usenix.org/conference/fast25/presentation/jeon>

This paper is included in the Proceedings of the
23rd USENIX Conference on File and Storage Technologies.

February 25–27, 2025 • Santa Clara, CA, USA

ISBN 978-1-939133-45-8

Open access to the Proceedings
of the 23rd USENIX Conference on
File and Storage Technologies
is sponsored by



AWUPF Rediscovered: Atomic Writes to Unleash Pivotal Fault-Tolerance in SSDs

Jiyune Jeon¹, Jongseok Kim¹, Sam H. Noh² and Euiseong Seo¹
¹ *Sungkyunkwan University* ² *Virginia Tech*

Abstract

From their inception, SSDs have ensured the atomicity of writes at the flash page level, guaranteeing their completion even during power failures. This functionality has been standardized as *Atomic Write Unit Power Fail* (AWUPF) in the NVMe standard. Despite SSDs providing AWUPF ranging from several to tens of KBs, there has been little effort on the host side to utilize this capability. For instance, if a transaction is smaller than the AWUPF size, leveraging AWUPF can eliminate the need for write-ahead logging or journaling. In this paper, we showcase how AWUPF reduces the overhead of host-side transactional writes through a light-weight crash consistency implementation for log-structured RAID (Log-RAID). Log-RAID manages the mapping of externally-exposed logical block numbers to their dynamically changing physical locations. Our approach bypasses journaling for updates of these mappings within the AWUPF limit, allowing direct writes instead. For larger updates, conventional journaling is applied. Additionally, our approach addresses the ordering issues between these two update paths. The evaluation of the proposed approach on Poseidon OS showed up to 3.6x improvement in random write performance.

1 INTRODUCTION

In modern flash memory SSDs, the smallest write unit is a page. A page write becomes externally visible only after the flash translation layer (FTL) updates its mapping, inherently giving SSDs atomic write characteristics [18]. The NVMe Specifications formalize this through the *atomic write unit power failure* (AWUPF) feature [5]. This mandatory feature ensures atomic writes when write sizes are within the AWUPF limit. In case of a power failure during a write operation, it guarantees that subsequent reads return the last successful write to the page. Despite SSDs supporting atomic writes up to several KB through AWUPF, there have been no attempts to utilize this feature on the host side so far.

File systems and DBMSs use journaling or write-ahead

logging (WAL) to maintain on-disk consistency even upon power failures or crashes. However, these approaches require double writes (first to the journal/WAL, then to the actual storage) [12, 30], leading to performance degradation and higher write amplification. Previous research has explored using SSD characteristics to bundle writes into atomic transactions and offload consistency mechanisms to SSDs [13, 21–23]; however, these approaches often demand changes to the FTL and new host interfaces.

We argue that transactional writes can be achieved using the existing AWUPF feature without requiring modifications to the SSD’s FTL or the host interface. Instead, by modifying the host-side software stack, the burden of processing a series of writes atomically can be offloaded to the SSD. This approach avoids unnecessary SSD changes while reducing the overhead of maintaining crash consistency.

However, traditional file systems need to modify multiple metadata entities simultaneously, such as inode bitmaps and inode entries, making it challenging to pack the updated metadata into the AWUPF size. Therefore, to showcase the effectiveness of the proposed concept, we utilize AWUPF to reduce journaling overhead in a Log-RAID system, a RAID architecture that writes data stripes sequentially in a log-structured manner [9, 10, 14, 19, 26]. Log-RAID minimizes random writes, improving performance especially on all-flash arrays (AFAs) while reducing write amplification. However, Log-RAID’s dynamic stripe mapping requires journaling to ensure crash consistency. Since mapping metadata updates typically occur in small, contiguous regions, the proposed concept can be applied with minimal modifications.

Specifically, we propose a dual update path strategy: metadata updates smaller than the AWUPF size directly modify the RAID metadata, while larger updates follow the traditional journaling path. We also introduce a mechanism to resolve order complications between these paths.

The proposed approach has been implemented in the Poseidon OS (POS) [1], an open-source storage operating system designed to provide an NVMe-oF interface in disaggregated AFA systems, and evaluated using FIO [7] and Filebench [27].

2 BACKGROUND AND RELATED WORK

2.1 Atomic Write Support in SSDs

SSDs send an acknowledgment to the host once data is committed to DRAM, even if it has not yet written to the NAND pages. Similarly, FTL mapping is updated in DRAM without immediately written to NAND [17, 25]. In these situations, to prevent data loss from unexpected crashes, SSDs use capacitors to power data transfer to flash memory [2–4, 25]. Under these mechanisms, SSDs can easily guarantee that data is written atomically for the size of a mapping unit by updating the mapping only after the flash page write is complete.

This inherent characteristic of SSDs is explicitly defined in the NVMe specification as AWUPF. AWUPF represents the size for atomic writes to non-volatile memory (NVM), and NAWUPF, which specifies this within a single namespace [5]. If a write command is within the AWUPF/NAWUPF size limit, the controller guarantees that subsequent reads will return either all the old data or all the new data, never a mix of partially written new data.

AWUPF is a feature that has been largely overlooked, and as such, it is not listed in most SSD specifications. However, by investigating the AWUPF of the SSDs we own through NVMe commands, we found that most SSDs provide a 4KB AWUPF, with some exceptions offering 512B, and in a unique case, a NAWUPF of 256KB¹. The SSD used in this study, as shown in Table 1, supports 4KB AWUPF.

Several studies have proposed interfaces that exploit SSD’s use of the FTL to treat a series of writes as a transaction by implementing atomic changes to the FTL mappings after all flash memory writes are completed [13, 21–23]. X-FTL includes a transactional page mapping table to handle multiple page writes as transactions [13]. This allows pages to be updated with the latest data while maintaining the previously committed pages and prevents pages invalidated by ongoing transactions from being erased by garbage collection. Additionally, X-FTL provides an extended SATA interface to support transactional writes. CFS is a file system that leverages SSDs offering such a transactional write interface [20]. Instead of using journaling, CFS guarantees crash consistency by utilizing SSDs’ transactional write feature.

Previous studies have focused on maintaining host-side write patterns that require atomicity by implementing FTL mapping structures and host interfaces supporting transactional writes across non-contiguous LBA regions, enabling SSDs to handle these writes as transactions. However, if a transaction fits within a single page, we can utilize the SSD’s existing page-level atomic write capability to guarantee transaction’s atomicity. This study is the first to show that, by adjusting the host-side metadata update mechanism, a significant portion of the crash consistency overhead can be offloaded to SSDs without altering the storage devices.

¹Western Digital Ultrastar DC ZN540

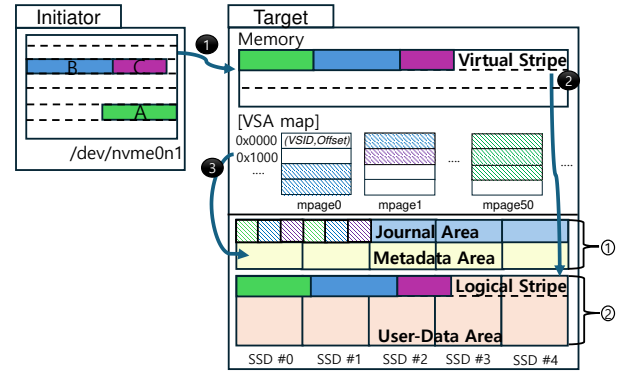


Figure 1: Log-RAID mapping structure of Poseidon OS

2.2 Log-RAID Design of Poseidon OS

Log-RAID writes data stripes sequentially in a log-structured manner [9, 10, 14, 19, 26], enabling sequential stripe writes even when the host’s write requests are random. This leads to reduced write-amplification-factor (WAF) and improved performance especially on AFAs [19]. However, unlike traditional RAID, where the physical location of a stripe with a specific LBA remains constant, in Log-RAID, the stripe’s physical location changes with each update. This requires maintaining a mapping between logical stripe numbers and their corresponding physical locations. Consequently, when a stripe is written, its mapping metadata must be updated. To prevent partial updates from power failures or crashes compromising RAID’s consistency, journaling updates to the mapping metadata is essential.

POS is an open-source AFA storage stack developed by Inspur and Samsung Electronics that binds multiple SSDs into a Log-RAID, providing virtual block devices to initiators via the NVMe-oF interface. To manage the logical-to-physical mappings for Log-RAID, POS divides the AFA into a metadata partition and a data partition. The metadata partition, denoted as ① in Fig. 1, is configured as RAID-10, and has two areas, one for journaling metadata updates and the other for storing the mapping metadata. Metadata is stored in 4 KB segments called *mpages*, which serve as the unit of writing to storage during checkpoints. The data partition, represented as ②, stores user data stripes according to Log-RAID principles.

In POS, a storage array is composed of multiple SSDs and provides several virtual block devices. To support this structure, the mapping scheme consists of two levels. The first-level mapping assigns a virtual block device’s block to a virtual address, which consists of a virtual stripe ID (VSID) and a logical offset within the stripe, as shown by ① in Fig. 1. The second-level mapping, represented by ②, maps the virtual address to its actual storage location, called the logical address, which includes a logical stripe ID (LSID) and a logical offset within the stripe. The logical stripes of an array are shared by all virtual block devices provided by that array. The RAID metadata area maintains the *virtual stripe address map* (VSA map) for the first-level mapping and the *stripe map* for

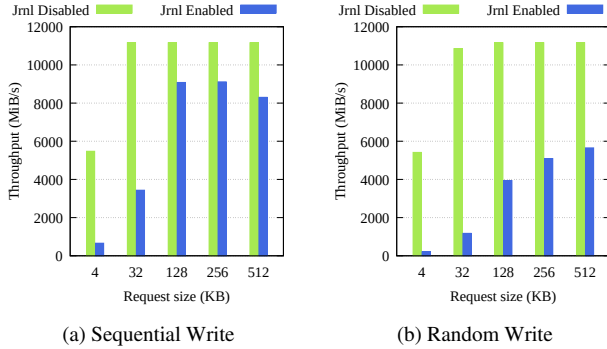


Figure 2: Write performance of Poseidon OS depending on the use of journaling across different write sizes

the second-level mapping. Using the LSID and logical offset, the actual SSD and the LBA in the SSD where the data is stored can be determined by performing division and modulus operations with the RAID chunk size.

Once user data is successfully written to the data partition, the VSA map is committed to the journal area. Unlike the widely-used JBD2 [24], which stores the entire image of updated pages in a commit, POS’s journal adopts a method that logs only the changes [8, 16]. The stripe map is committed to the journal area when the virtual stripe is filled. When the journal area becomes full, the committed transactions are checkpointed to the corresponding VSA map and stripe map. Like journaling in file systems, POS keeps the master copy of those metadata structures in main memory and first updates the in-memory VSA map with RAID metadata changes. When the journal area is full and a checkpoint is performed, the mpages that the commits have updates for are copied to the storage’s VSA map. Then, the journal area is reclaimed.

POS uses the Storage Performance Development Kit (SPDK) [29] for disk I/O and allows configuring the number of metadata processing reactors, which are threads responsible for I/O. With n reactors, the journal area is divided into n parts, and the journaling of a write request is handled by the (starting LBA of the request $\% n$)-th reactor.

Fig. 2 illustrates the performance impact of journaling on write requests of various sizes and patterns in the systems described in Table 1. When journaling is disabled, metadata is updated only in memory and periodically flushed to storage. As expected, enabling journaling significantly reduces performance due to the increased number of writes for metadata updates. This impact is more severe with smaller request sizes, as the number of commits rises for the same data volume. The performance difference between sequential and random writes is minimal without journaling, thanks to Log-RAID’s append-only write scheme. However, with journaling enabled, the performance gap widens further. This is because, during a checkpoint, multiple updates to a single mpage in sequential writes can be merged into a single mpage write, whereas in random writes, updates affect multiple mpages, resulting in a significantly higher number of mpage writes.

3 OUR APPROACH

3.1 Design Overview

The AWUPF feature guarantees atomicity for only a single write command. Therefore, metadata modifications requiring atomic guarantees must fit entirely within the AWUPF size’s contiguous space.

Recall that in POS, both VSA map and stripe map metadata requires journaling. Since AWUPF guarantees atomicity only for a write operation within its size limit and a single write operation cannot update both VSA and stripe maps, directly updating these metadata structures on storage could compromise crash consistency. The stripe map is updated only when a stripe is filled, while the VSA map is updated whenever an LBA is written, making stripe map updates less frequent and less impactful on journaling overhead. Therefore, we leverage AWUPF only for updating the VSA map by bypassing the journal layer and directly updating the mpages on storage.

In the VSA map, each entry in an mpage maps a 4 KB logical block of the virtual block device, hence one mpage contains mapping information for approximately 2 MB of virtual device blocks. Even if it is a sequential write, if the request size exceeds 2 MB, or the request is small but spans two mpages as in request A of Fig. 3a, the amount of metadata updated exceeds the AWUPF size, making the atomic update impossible. Thus, the write path must be adjusted depending on the request size. We choose to directly update the metadata area only if the size of the modified metadata does not exceed the AWUPF size; otherwise, the conventional journaling path is taken. This reduces, in proportion to the choice of the direct update path, both the number of committed transactions as well as checkpoint overhead. Note that performance in the worst case, which is when all sizes exceed the AWUPF size, is guaranteed to be no worse than the conventional method.

For example, in Fig. 1, the initiator requests three writes of different sizes: 10 KB (A), 12 KB (B), and 4 KB (C). These writes modify 3, 3, and 1 entries in the VSA map, respectively. However, for B, due to the starting offset of these entries, two mpages must be modified, which exceeds the AWUPF size, and thus B’s metadata changes are committed to the journal in the conventional manner. In contrast, both A and C only require an update of a single mpage, thus they directly update the VSA map in storage without going through the journal.

3.2 Ordering Complications and Resolutions

While the proposed approach described in the previous section may seem straightforward, ordering complications can arise. Specifically, consider a situation where the same mpage is updated via both journaling and direct paths, and a crash occurs. Two complications could emerge during recovery.

Fig. 3a shows the first case where write request A, which requires updates to two mpages, is performed first, followed by write request B, which updates only one mpage entry and thus directly updates mpage 1 bypassing journaling via



(b) The complete update to mpage 0 performed by write C is lost

Figure 3: Ordering complications between journaling and direct update

AWUPF. Suppose A modifies mpages 0 and 1 in memory and is committing to the journal but has not yet committed at the time of the crash, while B successfully updated mpage 1 in the storage metadata. Since the metadata updates of A have not been committed, neither A₀ nor A₁ should remain in storage in order to maintain consistency. However, such a crash could result in partial updates of A's metadata. Specifically, mpage 1 in memory reflects A₁ because the in-memory metadata is updated *before* journaling begins, and this will become persistent in storage because B directly updates mpage 1 on storage using its in-memory copy.

The second complication is shown in Fig. 3b where the commit for A is completed and mpage 0 is *directly* modified again, and then, a crash occurs. In this case, the recovery process may not be able to determine mpage 0's correct version due to its last entry. That is, as the last commit involving the entry in the journal says that it is A₀, the recovery process will replace its current mapping, which is C, with A₀ although A₀ is an old and stale value and C is the up-to-date one. To correctly maintain the latest version, we must retain the current mpage 0 by ignoring the log of A₀. At the same time, we need to recover mpage 1 for A₁, even though both mpages are part of the same commit transaction.

The key to resolving these ordering issues is recognizing that the latest version of an mpage is determined by the completion order of writes, not the order in which they are issued. This is because storage systems can execute commands out-of-order from the command queue, causing the issue order to differ from the completion order [28].

Specifically, to resolve the case in Fig. 3a, we modify the in-memory copy of an mpage *after* the journal commits the write operation, which is reflected in the gray shaded boxes coming after the blue boxes in Fig. 4. If a crash occurs at point ❶ in Fig. 4(a) or (c), that is, before the commit completes, B's

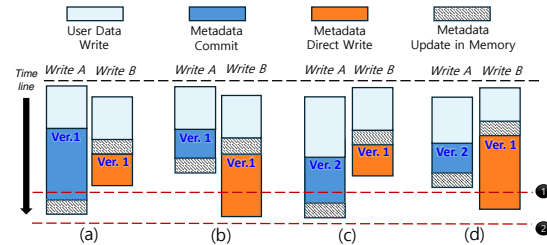


Figure 4: Resolving ordering complications between two write paths to the same mpage (its version is initially 0)

direct update of the metadata area's mpage will not include the changes made by A because they are not in the in-memory mpages yet; only the changes made by B will be persistent. In cases of Fig. 4(b) and (d), if a crash occurs at ❶, the changes from A will be applied during recovery through the commit log, but B's data will be discarded due to the AWUPF property, ensuring consistency.

To resolve the issue in Fig. 3b, we place a 64-bit header in each mpage, with 63 bits assigned as the version field and 1 bit as an *in-use* flag. The version of an mpage increments by 1 when a commit or a direct update begins, and the versions of updated mpages are also committed to the journal together with the change logs. During recovery, we compare the version values to determine the latest version when discrepancies occur between the commit log and the mpage stored on the metadata area. In addition to version, the *in-use* flag is needed for correct recovery. This flag is set for an mpage when a commit starts, and if a direct write targets an in-use mpage, the version is not incremented. Let us consider its effect using Fig. 4(a). We see in the figure that the direct update, B, starts after the commit for A begins but finishes first. In this case, if the in-use flag did not exist, the mpage's version will be 2 due to B. Consequently, the commit for A, which occurs after B, will be treated as outdated if a crash occurs at ❷ because of its lower version, being discarded during recovery. Thus, to prevent such situations, the in-use flag is first checked when a direct write is initiated on an mpage. If a direct write targets an in-use mpage, the version is not incremented, ensuring both writes are retained after recovery. Unlike direct updates, a commit transaction logs only the modified entries, allowing each transaction to be applied independently during recovery without depending on other commits. As a result, a commit increases the mpage's version even when it is in use, ensuring both commit transactions can be restored after a crash.

Next, we examine how crash recovery is handled. During crash recovery, the commit log for an mpage is replayed only if the version of the mpage in the commit transactions is greater than or equal to the version of the mpage stored in the metadata area. Obviously, the mpage in the commit log and the mpage in the metadata area can have the same version, but if they do, the same mapping entry could not have been modified. This is because the virtual file system (VFS) enforces page-level locks when writing dirty pages to storage [6, 11],

ensuring that simultaneous modifications to the same mapping entry at the same version cannot occur. Therefore, by merging the mpage in the metadata area with commit logs that have the same or a greater version, the latest version of the mpage is obtained. Additionally, there may be more than one commit log for the same mpage. In this case, recovery is performed in the order of commits based on the sequence number. The version of the mpage in the storage is not immediately updated during recovery. Instead, it is updated to the version of the last committed mpage after the recovery process is completed.

Furthermore, ② in Fig. 4 shows a situation where all write operations are completed, but a crash occurs before the checkpoint. In all four cases, the version of the commit log’s mpage is greater than or equal to the mpage’s version in the metadata area, thus all changes from the commit log are reflected in the mpage during recovery, ensuring the metadata area holds the most up-to-date copy.

Finally, if no crash occurs, the changes from both A and B in all cases in Fig. 4 will eventually be reflected in the metadata area after the checkpoint. This is because the journal checkpoint operation copies the *in-memory* mpages, which contain all changes from A and B, to the metadata area, rather than from the committed transactions.

4 Evaluation

4.1 Evaluation Environment

We compare the performance of the proposed approach with two configurations: the unmodified POS (denoted as *Journalled*) and the POS modified to directly write all updates to the VSA map in the metadata area (denoted as *Direct*), which lacks consistency guarantees. The server configuration used for evaluations is described in Table 1. Unless otherwise stated, the number of reactors for processing user data is set to 56, and the number for metadata is set to 20. The reported values are averages of five runs.

4.2 Micro-benchmark

While a metadata reactor performs a checkpoint, the other ones must wait to prevent them from modifying the shared in-memory VSA map for consistency. As a result, frequent checkpoints reduce journaling parallelism, making it difficult to parallelize journaling [15]. To evaluate how our proposed design affects POS scalability, we fix the FIO block sizes to 32KB and 1MB and vary the number of metadata reactors. FIO’s `numjobs` option is set to 20, and `ioengine` is configured to `SPDK`.

As shown at the bottom of Fig. 5, the performance of *Journalled* for 4KB random writes shows negligible improvement with increasing numbers of reactors due to high checkpoint overhead. In contrast, *Direct* exhibits continuous performance gains as the number of reactors increases, achieving up to 3.3 times higher throughput than *Journalled*. Since 4KB writes

Table 1: Specifications of the evaluation platform

Target Server	
Processor	Intel Xeon Gold 6336Y 24C/48T × 2 ea
Memory	DDR4 32GB RDIMM, 3200 Gbps × 16 ea
Storage Array	Samsung PM9A3 E1.S SSD 3.84 TB × 10 ea
Fabric	Mellanox Technologies ConnectX-5 100 GbE
OS Kernel	Linux 5.3.0
Initiator Server	
Processor	Intel Xeon Gold 6342 24C/48T × 2 ea
Memory	DDR4 32GB RDIMM, 3200 Gbps × 32 ea
Fabric	Intel E810-CQDA2 100 GbE
OS Kernel	Linux 5.4.0

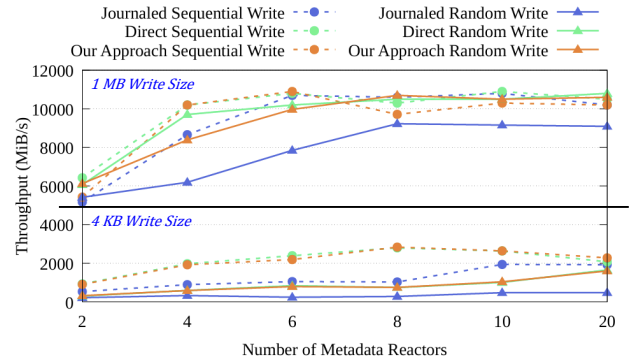


Figure 5: Write performance by metadata reactor count

modify only a single entry and all write requests rely on AWUPF for direct updates, *Our Approach* achieves results nearly identical to *Direct*. In sequential writes, both *Our Approach* and *Direct* show a slight performance decline after eight reactors due to increased contention on mpage accesses. However, *Journalled* improves as the number of reactors increases, benefiting from two batching effects. First, the commit overhead is reduced as multiple outstanding requests gathered during a single commit are written together in one operation. Second, during checkpoints, all updates for a single mpage are consolidated into a single mpage write, significantly reducing the number of mpage writes.

When the write size increases to 1MB, as shown at the top of Fig. 5, even for random writes, the performance of *Journalled* improves significantly as the write pattern becomes closer to sequential. However, *Our Approach* still outperforms *Journalled* by an average of 20% in random writes. This is because requests larger than a certain size are split due to the ext4 file system’s default stripe option, increasing the proportion of requests processed as direct writes and leading to a greater performance difference. *Our Approach* shows slightly lower performance than *Direct* since it still performs some journaling. In the case of sequential writes, the performance increase saturates at six reactors, after which all three configurations demonstrate comparable performance.

To further assess the impact of write size, we measure performance across different write sizes in FIO. As shown in Fig. 6a, the performance of sequential writes is similar across all three configurations due to previously discussed reasons. However, for 4KB random writes, as shown in Fig. 6b, *Our Approach* achieves 3.6 times better performance than *Jour-*

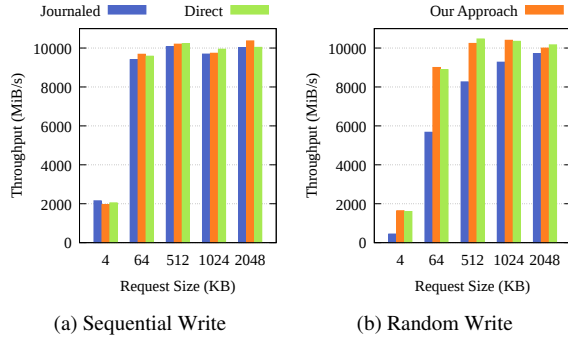


Figure 6: FIO write performance with varying request sizes

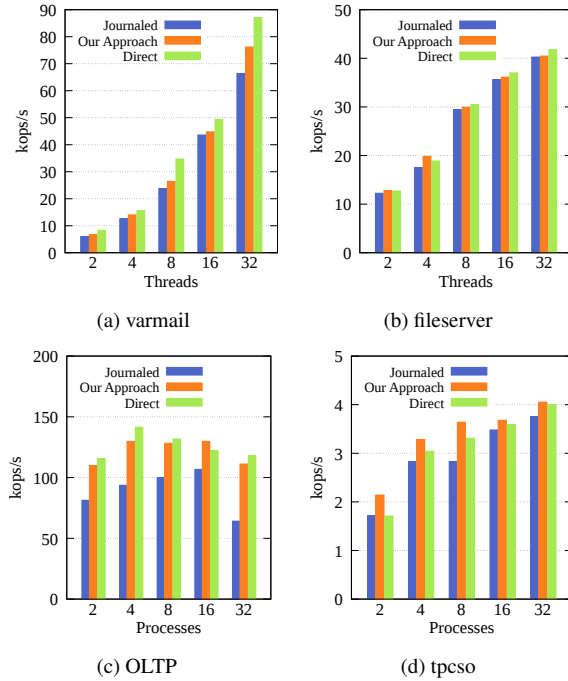


Figure 7: Performance for filebench workloads with varying number of writing threads (or processes)

naled. As the request size increases, the checkpoint overhead in *Journaled* decreases, reducing the performance gap between *Our Approach* and *Journaled*. Therefore, in the case of a 2MB write size, *Our Approach* frequently performs direct updates, making its performance closer to that of *Direct*.

4.3 Comprehensive Workloads

To evaluate our approach under various I/O patterns, we utilize varmail, fileserver, OLTP, and tpcso, which are filebench’s complex workloads conducting writes. We increase the number of preallocated files to make the initial file pool size approximately 100 GB.

As shown in Fig. 7, fileserver and varmail are similar in that they both repeatedly perform file creation, append, read, and delete operations. However, they differ in that fileserver includes large size sequential writes that write the entire file. In fileserver, requests larger than 512 KB account for 45% of the total number of requests and 94% of the total write

volume, whereas in varmail, they account for only 24% of the total requests and 57% of the total write volume. As a result, in fileserver, *Our Approach* and *Direct* were not significantly superior to *Journaled*, but in varmail, *Our Approach* showed 14% higher performance with 32 threads, and *Direct* showed 31% higher performance compared to *Journaled*. In varmail, small writes are performed through append operations after selecting a target file, which modifies consecutive entries. This results in a higher frequency of requests that modify consecutive mpages compared to pure random writes, like in FIO. Therefore, the proportion of requests handled by journaling increases in *Our Approach*, leading to performance results between those of *Direct* and *Journaled*.

In OLTP, the DB writers intensively issue small asynchronous random writes. As a result, *Our Approach* shows performance similar to *Direct* in all cases, with performance improvements ranging from 21% at 16 threads to 73% at 32 threads. In tpcso, the performance of only the DB writers, which perform small sequential writes, is measured by varying the number of them rather than measuring overall performance. Due to its low write intensity, the write performance improvement is smaller compared to OLTP.

Last but not least, when multiple mpages need to be updated, *Direct* writes each of them individually and the thread handling each update is determined by mpage’s index. Therefore, when updates span multiple mpages, they are handled by different threads, which can result in straggler threads, causing performance degradation. In contrast, *Our Approach* writes a journal commit that is significantly smaller. In addition, during a checkpoint, changes from multiple requests may be merged into a single mpage write. As a result, when circumstances align favorably, such as tpcso and some of the previous experiments, *Our Approach* can outperform *Direct*.

5 CONCLUSION

AWUPF is an intrinsic feature of SSDs that has, until now, been used solely for maintaining internal integrity. This study is the first attempt to leverage AWUPF for ensuring host-side consistency by shifting the atomicity guarantee of writes to the SSD, demonstrating its potential in applications requiring atomic writes. To showcase this potential, we proposed a lightweight journaling scheme for Log-RAID, achieving up to 3.6x performance improvement in random writes.

Our study focused on Log-RAID as it enables leveraging AWUPF with minimal modifications. However, file systems could also benefit from AWUPF if redesigned to ensure meta-data updates occur within the AWUPF size.

Acknowledgements

This research was supported by Samsung Electronics, the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2023-00321688) and NSF grant (2312785).

References

- [1] Poseidon OS. <https://github.com/poseidonos/poseidonos>.
- [2] How Micron SSDs handle unexpected power loss. White paper, Micron Technology, Inc., 2014. <https://www.micron.com/content/dam/micron/global/public/products/white-paper/ssd-power-loss-protection-white-paper-lo.pdf>.
- [3] Power loss imminent(PLI) technology. White Paper 330275-001US, Intel Corporation, 2014. <https://www.intel.com/content/dam/www/public/us/en/documents/technology-briefs/ssd-power-loss-imminent-technology-brief.pdf>.
- [4] Unexpected power loss protection. White paper, SanDisk LLC, 2015. https://www.reactivedata.com/Downloads/Sandisk_Unexpected_Power_Loss_Protection.pdf.
- [5] NVM express NVM command set specification 1.0c, October 2022. <https://nvmexpress.org/wp-content/uploads/NVM-Express-NVM-Command-Set-Specification-1.0c-2022.10.03-Ratified.pdf>.
- [6] File systems locking (version 6.10.0), 2024. <https://www.kernel.org/doc/Documentation/filesystems/locking.rst>.
- [7] Jens Axboe. FIO-flexible IO tester. <https://github.com/axboe/fio>.
- [8] Srivatsa S Bhat, Rasha Eqbal, Austin T Clements, M Frans Kaashoek, and Nickolai Zeldovich. Scaling a file system to many cores using an operation log. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, pages 69–86, 2017.
- [9] Tzi-cker Chiueh, Weafon Tsao, Hou-Chiang Sun, Ting-Fang Chien, An-Nan Chang, and Cheng-Ding Chen. Software orchestrated flash array. In *Proceedings of International Conference on Systems and Storage (SYSTOR)*, pages 1–11, 2014.
- [10] John Colgrove, John D Davis, John Hayes, Ethan L Miller, Cary Sandvig, Russell Sears, Ari Tamches, Neil Vachharajani, and Feng Wang. Purity: Building fast, highlyavailable enterprise flash storage from commodity components. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, pages 1683–1694, 2015.
- [11] Richard Gooch. Overview of the Linux virtual file system (version 6.10.0), 2024. <https://www.kernel.org/doc/Documentation/filesystems/vfs.rst>.
- [12] Woon-Hak Kang, Sang-Won Lee, Bongki Moon, Yang-Suk Kee, and Moonwook Oh. Durable write cache in flash memory SSD for relational and NoSQL databases. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*, pages 529–540, 2014.
- [13] Woon-Hak Kang, Sang-Won Lee, Bongki Moon, Gi-Hwan Oh, and Changwoo Min. X-FTL: transactional FTL for SQLite databases. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, pages 97–108, 2013.
- [14] Jaeho Kim, Kwanghyun Lim, Youngdon Jung, Sungjin Lee, Changwoo Min, and Sam H Noh. Alleviating garbage collection interference through spatial separation in all flash arrays. In *Proceedings of the 2019 USENIX Annual Technical Conference (ATC)*, pages 799–812, 2019.
- [15] Jongseok Kim, Cassiano Campos, Joo-Young Hwang, Jinkyu Jeong, and Euiseong Seo. Z-Journal: Scalable per-core journaling. In *Proceedings of the 2021 USENIX Annual Technical Conference (ATC)*, pages 893–906, 2021.
- [16] Youngjin Kwon, Henrique Fingler, Tyler Hunt, Simon Peter, Emmett Witchel, and Thomas Anderson. Strata: A cross media file system. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, pages 460–477, 2017.
- [17] Hyeon Gyu Lee, Juwon Lee, Minwook Kim, Donghwa Shin, Sungjin Lee, Bryan S Kim, Eunji Lee, and Sang Lyul Min. SpartanSSD: A reliable SSD under capacitance constraints. In *Proceedings of IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*, pages 1–6. IEEE, 2021.
- [18] Youyou Lu, Jiwu Shu, Jia Guo, Shuai Li, and Onur Mutlu. LightTx: A lightweight transactional design in flash-based SSDs to support flexible transactions. In *Proceedings of the 31st International Conference on Computer Design (ICCD)*, pages 115–122. IEEE, 2013.
- [19] Teng Ma, Zhitao Li, and Ning Liu. Log-ROC: Log structured RAID on open-channel SSD. In *Proceedings of the 40th International Conference on Computer Design (ICCD)*, pages 332–335. IEEE, 2022.
- [20] Changwoo Min, Woon-Hak Kang, Taesoo Kim, Sang-Won Lee, and Young Ik Eom. Lightweight application-level crash consistency on transactional flash storage. In *Proceedings of the 2015 USENIX Annual Technical Conference (ATC)*, pages 221–234, 2015.

- [21] Xiangyong Ouyang, David Nellans, Robert Wipfel, David Flynn, and Dhabaleswar K Panda. Beyond block I/O: Rethinking traditional storage primitives. In *Proceedings of the 2011 IEEE 17th International Symposium on High Performance Computer Architecture (HPCA)*, pages 301–311. IEEE, 2011.
- [22] Sunhwa Park, Ji Hyun Yu, and Seong Yong Ohm. Atomic write FTL for robust flash file system. In *Proceedings of the Ninth International Symposium on Consumer Electronics (ISCE)*, pages 155–160. IEEE, 2005.
- [23] Vijayan Prabhakaran, Thomas L Rodeheffer, and Lidong Zhou. Transactional flash. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, volume 8, 2008.
- [24] Stephen C Tweedie et al. Journaling the Linux ext2fs filesystem. In *Proceedings of the Fourth Annual Linux Expo*. Durham, North Carolina, 1998.
- [25] Kristian Vättö. Power loss protection in SSDs: how SSDs are protecting data integrity. White Paper WHP-SSD-POWERLOSSPROTECTION-FEB16J, Samsung Electronics Co., Ltd., 2016. https://image-us.samsung.com/SamsungUS/b2b/resource/2016/07/08/b2b_resource_WHP-SSD-POWERLOSSPROTECTION-r1-JUL16J.pdf.
- [26] Qiuping Wang and Patrick PC Lee. ZapRAID: Toward high-performance RAID for ZNS SSDs via zone append. In *Proceedings of the 14th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys)*, pages 24–29, 2023.
- [27] Andrew Wilson. The new and improved filebench. In *Proceedings of the 6th USENIX Conference on File and Storage Technologies (FAST)*, 2008.
- [28] Youjip Won, Jaemin Jung, Gyeongyeol Choi, Joontaek Oh, Seongbae Son, Jooyoung Hwang, and Sangyeun Cho. Barrier-enabled IO stack for flash storage. In *Proceedings of the 16th USENIX Conference on File and Storage Technologies (FAST)*, pages 211–226, 2018.
- [29] Ziye Yang, James R Harris, Benjamin Walker, Daniel Verkamp, Changpeng Liu, Cunyin Chang, Gang Cao, Jonathan Stern, Vishal Verma, and Luse E Paul. SPDK: A development kit to build high performance storage applications. In *Proceedings of the 2017 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, pages 154–161. IEEE, 2017.
- [30] You Zhou, Qiulin Wu, Fei Wu, Hong Jiang, Jian Zhou, and Changsheng Xie. Remap-SSD: Safely and efficiently exploiting SSD address remapping to eliminate duplicate writes. In *Proceedings of the 19th USENIX Conference on File and Storage Technologies (FAST)*, pages 187–202, 2021.